

Past Paper Questions

1) In what situation we use Cross Compilers?

⇒ Cross Compilers are used to compile code on one platform for it to run on a different platform, often when they have different architectures or OS. This is helpful for embedded systems, different OS, mobile devices, and more.

2) How a Compiler is categorized?

(1) The source language that is compiled.

(2) The target language it is generated.

(3) The implementation language that is used for writing the compiler.

(3) Difference b/w Compiler & Cross Compiler?

Compiler

⇒ A software that translates the computer code written in high-level programming

Cross Compiler

⇒ A software that can convert create executable code for platform other than one on

language to machine language which the compiler is running.

Q. Difference between compiler and interpreter?

Solved already.

Q. Write the name of phases of compiler in sequence?

Solved already.

Q. Define Cross Compiler?

Solved already.

Q. Draw the structure of a Compiler.

Solved already.

Q. How hierarchical analysis take place?

⇒ In hierarchical analysis

characters or tokens are grouped hierarchically into nested collections with collective meaning.

Q. How to analyze the source program?

⇒ Analysis of the source program is divided into three parts.

(1) Linear analysis:

⇒ In which the stream of characters making up the source program is read from left to right and grouped into tokens that are sequences of characters having collective meaning. Also called lexical analysis.

(2) Hierarchical Analysis

⇒ In which characters or tokens are grouped hierarchically into nested collections with collective meaning.

(3) Semantic Analysis:

⇒ In which certain checks are performed to ensure that the components of a program fit together

meaningfully.

Q. What is Assembler?

⇒ An assembler is a component used in language processing system that converts assembly language code, generated by the compiler, into a machine code ^{or relocatable code} that a computer CPU can execute.

⇒ It completes the process of translating human readable code into instructions the computer understands.

Q. Why we use cross compiler?

Solved already?

Q. What is the responsibility of loader?

Solved already.

Q. Is it possible to use a common back end for different frontends?

⇒ Yes, that's likely. But subtle differences between languages can result in different ^{from} similar looking source.
⇒ it's rare that the front-end will give the back end exactly the same inputs.

Q. What Compiler Construction Tools?

⇒ The Compiler writer can use some specialized tools that help in implementing various phases of a compiler.
These tools assist in the creation of entire compiler or its parts.

⇒ Some commonly used are:

- Scanner generator
- Parser generator
- Syntax directed translation engine
- Automatic code generators

◦ Data flow analysis engine

◦ Compiler Construction tool kit

Q. Name the layers associated with front end? Solved already

Q. What are principles of a Compiler?

⇒ The Compiler must preserve the meaning of the program being compiled.
⇒ Improve the source code in some way.

Other issues (depending on the setting):

- ⇒ Speed (of compiled code).
- ⇒ Space (size of compiled code)
- ⇒ Feedback (information provided to the users)
- ⇒ Debugging (transformations obscure the ^{source code} _{is target})
- ⇒ Compilation time efficiency (fast or slow _{Compiler?})

Q. What is the analysis-synthesis model of compilation?

These are two parts of compilation:

⇒ Analysis: determines the operations implied by the source program which are recorded in a tree structure.

⇒ In analysis phase, an intermediate representation is created from the given source program.

⇒ Synthesis: takes the tree structure and translates the operations there into the target program.

Q. What Qualities would you want in a Compiler?

- ⇒ Generate correct code (fast and precise).
- ⇒ Generates fast code
- ⇒ Conforms to the specifications of the input language
- ⇒ Copes with essentially arbitrary I/P size, variables etc.
- ⇒ Compilation time
- ⇒ Good diagnostic
- ⇒ Consistent optimisations
- ⇒ Works well with the debugger.

Q. What is the front end?

- ⇒ Front end performs the analysis of the source language.
- ⇒ Recognises legal and illegal programs and reports error.
- ⇒ "Understands" the I/P program and collects its semantics in an IR.
- ⇒ Produces IR and shapes the code for the back-end.
- ⇒ Much can be automated.
- ⇒ Typically, front end is $O(n)$.

Q. What is back-end?

- ⇒ Back end does the target language synthesis.
- ⇒ Chooses instructions to implement each IR Operation.
- ⇒ Translates IR into target code.
- ⇒ Needs to conform with system integrator.
- ⇒ Automation has been less successful.
- ⇒ Back end is NP Complete.

Q. What is language Converter?

- ⇒ Language Converter also known as language translator.
- ⇒ It is a program which is used to translate instructions that are written in source code to object code.
- ⇒ Convert from high-level language or assembly language into machine language.
- ⇒ Generally, there are three types:
 - Compilers
 - Interpreters
 - Assemblers

Q. How grouping different phases of compiler creates problem?

When grouping phases into one pass the entire program has to be kept in memory to ensure proper information flow to each phase because one phase may need information in a different order than the information produced in previous phase. The source program or target program differs from its internal representation.

Q. What are the benefits of language translators?

Some benefits are:

- ⇒ Cross language compatibility
- ⇒ Code reusability
- ⇒ Platform independence
- ⇒ Rapid development
- ⇒ Migration & legacy systems
- ⇒ Access to libraries
- ⇒ Language transition
- ⇒ Optimization & refactoring
- ⇒ Learning curve

Example: OF Compiles Construction Phases:-

2: Translate the given statement into Phase wise with ILP and OLP of each phase.

$$\text{Sum} = \text{initial} + \text{value} * 10$$

Source program

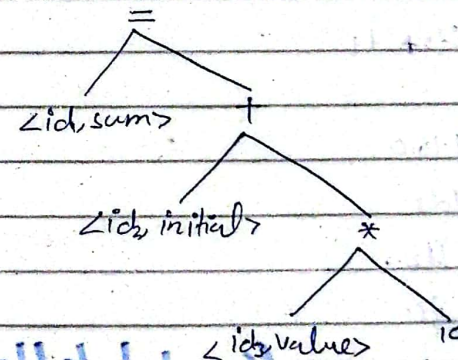
$$\text{Sum} = \text{initial} + \text{value} * 10$$

(1)

$$\langle \text{id}, \text{sum} \rangle \leftarrow \langle \text{id}, \text{initial} \rangle \leftarrow \langle + \rangle \leftarrow \langle \text{id}, \text{value} \rangle$$

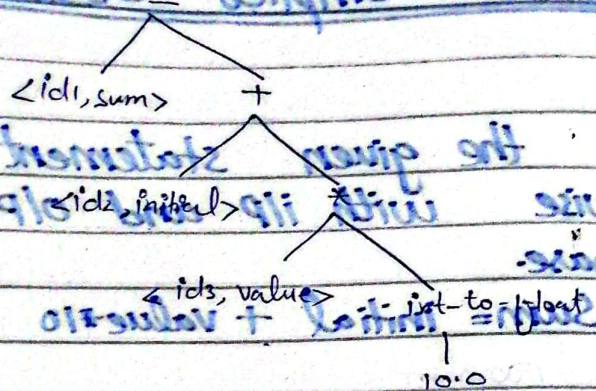
$$\langle * \rangle \leftarrow \langle 10 \rangle$$

(2)



(old id, value)

(3)



(4)

t1 = int-to-float(10.0)
t2 = id3 * t1
id1 = t2

(5)

t1 = id3 * 10.0
id1 = id2 * t1

(6)

LDA #10.0
MUL Id3
ADD Id2
STA id1

1	Sum
2	initial
3	Value

(Symbol table).

Q. Define phases of a compiler?

⇒ Compiler is a program, to understand the design of the compiler, it is logically divided into various sub-programs called modules. These subprograms or modules are also called phases.

⇒ Each phase or module accepts one representation of the program as the input and generates another representation of the program as the o/p.

Q. Define pass?

• A group of one or more phases of a compiler that performs analysis or synthesis of the source program is called as a pass of the compiler.

Q. What is symbol table?

⇒ ^{Runtime} Data structure created and maintained by compiler in order to store information about occurrences of various entity such as variable and functions names.

o Objects o Classes o interfaces etc.

⇒ Information inside the symbol table is gathered from analysis phase which in turn used synthesis phases in order to generate target code.

Usage:

Phase-1: Creates entries for identifiers.

Phase-2: Add information regarding attributes (like type, scope, line of reference etc.)

Phase-3: Using a valid information context semantics and updates the symbol table.

Phase-4: Available information helps in adding temporary variables information.

Phase-5: Available information is used for machine dependent optimization.

Phase-6: Generates the target code using address information of identifiers.

Entries in Symbol table:

- o Name
- o Type
- o Size
- o Dimension
- o Line of declaration
- o Line of usage
- o Address
- o Constant

- Symbol table only uses in synthesis phase
symbol table need

to support multiple declaration of the same identifiers within program

- Based on scope types of symbol table

- o Global ST
- o Scope ST

Example:

(1) int count;

(2) char x[] = "Neso Academy";

Name	Type	Size	Dimension	LOD	LOU	Address
count	int	1	0	---	---	---
x	char	12	-	---	---	---

Operations:-

- 1) Non-Blocking Structured language:-
 - o Contains the single instance of the variable declaration.
 - o Did not use any language blocks.

Operations:

(1) Insert()

(ii) Lookup()

(2) Block-Structured Languages

⇒ Blocks are specified by curly braces.
⇒ Variable declaration may be multiple

times.

- Insert ^(is done once)
- Lookup ^(is done many times)
- Set
- Reset

Why look up?

⇒ To determine:

- if the name exists in the table.
- if it is declared before it is being used.
- if name is initialized.
- if the name declared multiple time.
- if the name is used in scope.